Using Enums and Literal Types

TypeScript is nothing if not chock full of many different ways to specify a type that is limited to a few values. We've just looked at literal types, but those are effectively just a subset of what you can do in TypeScript using an enumerated type, or enum.

Defining Enums

TypeScript enums are very flexible and allow you to do a lot of things that seem, frankly, to be rather pointless. I'll try to minimize talking about the pointless parts.

At the simplest, you can define an enum in TypeScript just by providing a list of values for it. So if we wanted to convert our TicketStatus type to an enum, we could do this:

chapter_13/02/app/javascript/contexts/venue_types.ts

```
export enum TicketStatus {
  Unsold,
  Held,
  Purchased,
  Refunded,
  Invalid,
}
```

We can then use these statuses as TicketStatus.Unsold, TicketStatus.Held, and so on. By convention, the individual elements of the enum are capitalized because they are also types.

However, if we make this change, we get a lot of downstream compiler errors. Every time we assign a string to a TicketStatus or compare a string to a TicketStatus, we need to replace that string with one of the enum values.

For example, in our pre-enum seat.tsx file, we currently have: